

university of groningen

Analysis and Optimisation of CloverLeaf on a Kunpeng 920 Node

Performance Engineering, Group 11

Jort Oosterveld Sergiu-Adrian Preda

Faculty of Science and Engineering, University of Groningen

October 2026

Baseline

The application and the machine

CloverLeaf (reference, MPI + OpenMP)

- 2D compressible Euler, Lagrangian–Eulerian step
- 13 kernels, each a ± 1 stencil loop nest over the grid
- we optimise the *C kernels*; Fortran kernels only as reference
- benchmark **bm3840**: 3840^2 cells, 87 steps, $\approx$25 arrays of 118 MB ($\approx$3 GB working set, $25\times$ L3)

HAICGU node

- $2\times$ Kunpeng 920 (ARMv8, 2.6 GHz): 128 cores, 4 NUMA domains
- 16 channels DDR4-2666, 128 GB, L3 $2\times$64 MB
- GCC 14.1, OpenMPI 5.0, PAPI 7.1, Linux `perf`
- `-O3 -march=native -funroll-loops -fopenmp`
- layout "$r \times t$": r ranks $\times$ t threads, one thread per core, ranks pinned

Question of the project: how far is the baseline from what the hardware allows, and what does it take to close the gap?

Methodology: three independent sources, one convention

1. CloverLeaf's own profiler: wall clock per phase
2. PAPI per kernel (`papi_prof`, one event set per thread): IPC, instruction mix, stalls
3. Linux `perf` and the DDR-controller PMU: **bytes that cross the memory interface**, read around every run

Protocol. 5 repetitions, median and min–max, every one under the DDR counters: time and bytes from the same run. All bandwidths are *hardware bytes* / wall clock, for STREAM and for the application alike.

Checks of the tools

- PAPI build costs $<1\,\%$; kernel times add up to the wall clock within $1\,\%$
- DDR counters calibrated on STREAM: $32\,\mathrm{B}$ per count, no read-for-ownership on streaming stores
- placement verified: busiest controller group $= 1.00\times$ the mean

Correctness gate

`PASSED` on bm3840; deviation of the long deck `clover_bm.in` reproduced to $10^{-6}\,\%$

Where the time goes

kernel	time share (%) 1 core	128 cores	IPC 64×2	stall (%) 64×2
`advec_mom`	28.6	**44.0**	0.27	91
`advec_cell`	19.4	**19.4**	0.28	91
`pdv` (2 calls)	17.6	9.6	0.21	92
`calc_dt`	11.8	4.8	0.50	81
`ideal_gas`	7.4	4.7	0.31	87
`viscosity`	5.7	2.2	0.99	62
`accelerate`	5.2	4.1	0.27	91
`reset` + `revert`	1.6	4.5	0.10	96
other (3)	2.7	6.7	–	–

bm3840, PAPI; 128 cores = 1×128; stall = back-end stall cycles

- advection = 63 % of the node-level time; `advec_mom` scales worst (14× on 128 cores)
- at 64×2: IPC 0.1–0.5, **81–96 % back-end stalls**: the cores wait for memory
- `calc_dt`, `ideal_gas`, `viscosity` run *scalar*: `sqrt` stays a libm call without `-fno-math-errno`
- advection upwind loops: data-dependent indices, only 12–16 % vector instructions
- 12.5 % of all cycles in the OpenMP barrier (one per worksharing loop)

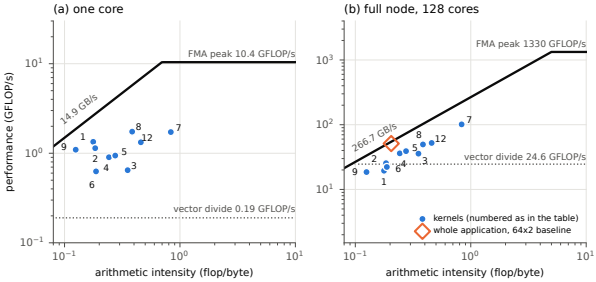
Ceilings, byte model and roofline

Measured ceilings

- STREAM triad, 128 threads: **266.5 GB/s** hardware bytes
- FMA peak **1330 GFLOP/s**: ridge at **5 flop/B**

Analytic byte model (arrays read and written per loop nest, write-allocate charged)

- 451 flop, 2186 B per cell and step: **0.21 flop/B**
- predicts 2.80 TB for bm3840; measured **2.83 TB** ($+1\,\%$)
- reads $= 1.34\times$ demand: the C stores pay a read-for-ownership



university of groningen

Baseline: already at the memory ceiling

kernels	layout	wall (s)	TB	% of roof
C	**64×2**	**11.32**	2.83	**94**
C	32×4	11.46	2.79	91
C	128×1	11.66	2.95	95
C	4×32	13.00	2.87	83
C	1×128	26.91	2.99	42
C	1×128, THP off	13.31	2.83	80
Fortran	64×2	9.33	**2.36**	95

bm3840, median of 5; roof = 266.5 GB/s hardware bytes

Consequence

No headroom at the current byte count: every further gain has to come from **moving fewer bytes**.

- baseline 64×2: 250 GB/s over the whole run, start-up included: **94 % of the hardware ceiling**
- memory floor at the current bytes: $2.83\,\text{TB}/266.5\,GB/s = \mathbf{10.6\,s}$; the baseline is within 7 %
- only 1×128 cares about THP: first touch puts whole arrays on one NUMA domain ($1.66\times$ imbalance)
- Fortran is 18 % faster at the *same* bandwidth: it moves 0.47 TB fewer bytes

Kernel improvements

Plan: five byte reductions, predicted with the byte model

change	DRAM bytes	cumulative	floor (s)
(1) `advec_mom`: direction-only helper loops once per sweep, not once per velocity component	−12.8 %	−12.8 %	9.2
(2) `advec_cell`: four temporaries in registers instead of four full-mesh arrays	−5.9 %	−18.7 %	9.9
(3) dead stores `post_vol` / `pre_vol`: written over the haloed grid, never read	−4.4 %	−21.6 %	10.1
(4) Reset / Revert: swap the field allocations instead of copying 118 MB per field	−6.6 %	−28.2 %	9.8
(5) `calc_dt`: fuse the min reduction into the compute loop	−1.1 %	**−29.3 %**	**7.4**
(6) streaming stores (no read-for-ownership), upper bound	−22.8 %	−41.4 %	–

- selection: the three kernels with the largest gain in the model; (3) and (5) fell out while reading the code
- every change is an edit of the model's loop table, so prediction and measurement share one convention; time-only items (`nowait`, `-fno-math-errno`, branch-free upwind loop) come after the bytes

The changes in code (1/2)

(2) `advec_cell`: temporaries in registers

```
/* before: 4 full-mesh stores + reloads per sweep */
pre_mass[i]  = density1[i]*pre_vol[i];
post_mass[i] = pre_mass[i] + mass_flux_x[i] - mass_flux_x[i+1];
post_ener[i] = (energy1[i]*pre_mass[i] + ener_flux[i] - ener_flux[i+1]) /
       post_mass[i];
advec_vol[i] = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
density1[i]  = post_mass[i] / advec_vol[i];
energy1[i]   = post_ener[i];

/* after: same arithmetic, same order, no DRAM traffic */
double pre_mass_s  = density1[i]*pre_vol[i];
double post_mass_s = pre_mass_s + mass_flux_x[i] - mass_flux_x[i+1];
double post_ener_s = (energy1[i]*pre_mass_s + ener_flux[i] - ener_flux[i
       +1]) / post_mass_s;
double advec_vol_s = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
density1[i] = post_mass_s / advec_vol_s;
energy1[i]  = post_ener_s;
```

(3) Dead stores. Delete the `post_vol` store in sweep 1 of `advec_cell` and the `pre_vol` store in `advec_mom`; each was a full-mesh write plus its read-for-ownership, once per call.

(4) Reset / Revert without copies

```
! reset_field.f90, C path: exchange the allocations,
! reset_field_kernel_c is no longer called
CALL swap_field(field%density0, field%density1)
CALL swap_field(field%energy0,  field%energy1)
CALL swap_field(field%xvel0,    field%xvel1)
CALL swap_field(field%yvel0,    field%yvel1)

SUBROUTINE swap_field(a,b)   ! three MOVE_ALLOCs: O(1)
  CALL MOVE_ALLOC(a,t); CALL MOVE_ALLOC(b,a); CALL MOVE_ALLOC(t,b)
END SUBROUTINE

! PdV.f90: the C corrector rewrites density1/energy1
! over the whole range that revert would copy
IF ( predict .AND. .NOT. use_C_kernels ) CALL revert()
```

Before: 4 fields $\times$ 118 MB read, written and read-for-ownership, every step, for a copy no kernel needs at a fixed address.

The changes in code (2/2)

(1) `advec_mom`: split prep and velocity part

```
! before: the kernel ran for xvel AND yvel; its first four
! loops depend only on the sweep direction and rebuilt
! identical arrays the second time (8 arrays streamed twice)
DO tile=1,tiles_per_chunk
  CALL advec_mom_prep_driver(tile,direction,sweep)  ! once per sweep
  CALL advec_mom_driver(tile,xvel,direction,sweep)  ! velocity part
  CALL advec_mom_driver(tile,yvel,direction,sweep)
ENDDO
```

(5) `calc_dt`: fused reduction

```
/* before: store dt_min[i] per cell, reload it in a second loop */
#pragma omp for reduction(min:dt_min_val)
for (k...) {
#pragma omp simd reduction(min:dt_min_val)
  for (j...) {
    cell_dt_limits(j,k, ..., &dtct,&dtut,&dtvt,&dtdivt);
    dt_min_val = MIN(dt_min_val, MIN(dtct,MIN(dtut,MIN(dtvt,dtdivt))));
  } }   /* dt_min[] never written; the limiting cell is
           located only on the failure path */
```

Time-only changes, done in `advec_mom`

- `-fno-math-errno -fno-trapping-math`: vector `sqrt`, if-converted loops; worth 3.6 % but they break the gate: dropped (correctness slide)
- `nowait` between loops over the same rows with `schedule(static)`
- branch-free upwind loop: load all four candidates, select instead of indirect loads (backup slide)

(6) Streaming stores: left to the hardware. No portable non-temporal store for AArch64 in GCC; the output loops keep the shape the write-streaming detector needs.

Results

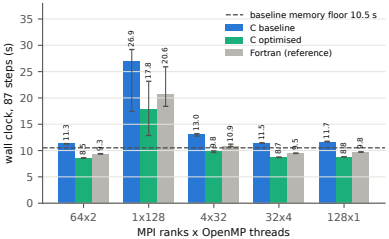
Ladder at 64×2: the wall clock follows the bytes

build	contents	wall (s)	TB	bytes	time
base	baseline source	11.31	2.83	–	–
flags	the two flags only	11.28	2.85	+0.7 %	−0.2 %
cell	flags + (2) (3) (4)	9.92	2.49	−12.1 %	−12.3 %
mom	flags + (1) (3) (5)	9.54	2.41	−14.9 %	−15.7 %
opt	all, with the flags	8.25	2.04	−27.8 %	−27.0 %
optnf	all, baseline flags	8.56	2.05	−27.4 %	−24.3 %
pdv	optnf + PdV scalar	**8.25**	**2.01**	**−28.8 %**	**−26.5 %**

jobs 34938, 35003 (optnf), 35016 (pdv): one node, 5 repetitions per build (spread ≤ 1 %), DDR counters on

- the lines add up: −28.8 % bytes against the −30.8 % **the model predicted**
- every build sustains 240–252 GB/s: still on the roof, so time $\propto$ bytes
- the flags: 0 % on the baseline, 3.6 % on the optimised kernels, and they break the gate; **pdv** is the final build
- **1.37×** at 64×2: 11.32 s to 8.26 s

Every layout, every kernel



layout	64×2	32×4	128×1	4×32	1×128
speed-up	1.37	1.36	1.36	1.37	1.33

kernel	base (s)	pdv (s)	ratio
`advec_mom`	4.19	2.62	**1.60**
`advec_cell`	2.42	1.67	**1.45**
`pdv (both passes)`	1.32	1.17	1.13
`calc_dt`	0.66	0.62	1.05
`ideal_gas`	0.58	0.54	1.06
`reset + revert`	0.57	–	gone
`accel. + flux`	0.95	1.00	0.95

PAPI region times at 64×2, jobs 34798 and 35017 (pdv)

- the same quarter of the traffic gone at every layout, the same speed-up
- optimised C is now **faster than the Fortran reference at every layout**
- the application moved *up* the roof, not off it: 91 % of the ceiling, 9 % above the floor of its own bytes

university of groningen

Correctness, and what is still open

Gate on `clover_bm.in` (960^2, 2955 steps)

- the ladder's baseline binary reproduces the archived reference (0.0057 %) at every layout
- every build with the two compiler flags, flags-only included, gives *one and the same* other value (0.038 %): the flags alone move it (GCC's multiply-add contraction; bisect, job 34936)
- **optnf** (kernel changes, baseline flags) and **pdv** (optnf + PdV scalar) reproduce the reference at every layout (7 of 7 each)
- the flags were worth 3.6 % on top (vectorised `sqrt` in `calc_dt`, `ideal_gas`): a measured trade-off, decided for reproducibility

Limitations and next steps

- still on the bandwidth roof: 91 % of 266.5 GB/s, floor 7.6 s; `accelerate`, `flux_calc`, `viscosity` untouched: the next bytes to remove
- read-for-ownership of the C stores: up to -23 % more bytes with explicit streaming stores
- `nowait` and branch-free upwind loop still open in `advec_cell`: worth at most 9 %

Conclusions

1. The C kernels are memory-bound (0.2 flop/B against a 5 flop/B ridge) and the baseline already sustains 94 % of the 266.5 GB/s hardware ceiling: **only fewer bytes can make it faster**.
2. An analytic byte model, validated by the DDR counters to 1 %, predicted five changes at -29 % of the traffic, six at -31 %; measured: -28.8 %.
3. The wall clock followed the bytes: **1.37× at 64×2, 1.33–1.37× at every layout**; the optimised C build beats the Fortran reference everywhere and passes the gate.
4. The two compiler flags are worth 3.6 % on the optimised kernels but break bitwise reproducibility of the reference: dropped, as a measured trade-off. The next lever is the read-for-ownership of the stores, not instruction-side work.

Questions?

Backup

Backup: per-kernel counters at 64×2

kernel	time (s)			IPC		back-end stall (%)	
	base	pdv	ratio	base	pdv	base	pdv
advec_mom	4.19	2.62	1.60	0.27	0.28	91	89
advec_cell	2.42	1.67	1.45	0.28	0.36	91	88
pdv_correct	0.71	0.63	1.13	0.20	0.23	92	92
calc_dt	0.66	0.62	1.05	0.50	0.55	81	80
pdv_predict	0.61	0.54	1.13	0.22	0.24	92	90
ideal_gas	0.58	0.54	1.06	0.31	0.32	87	87
accelerate	0.56	0.61	0.93	0.27	0.26	91	91
flux_calc	0.39	0.39	1.01	0.16	0.16	95	95
reset_field	0.38	–	–	0.10	–	96	–
viscosity	0.32	0.33	0.98	0.99	0.96	62	63
revert	0.19	–	–	0.10	–	96	–
update_halo	0.04	0.04	1.05	0.57	0.60	42	41
field_summary	0.03	0.03	1.03	0.42	0.43	85	85

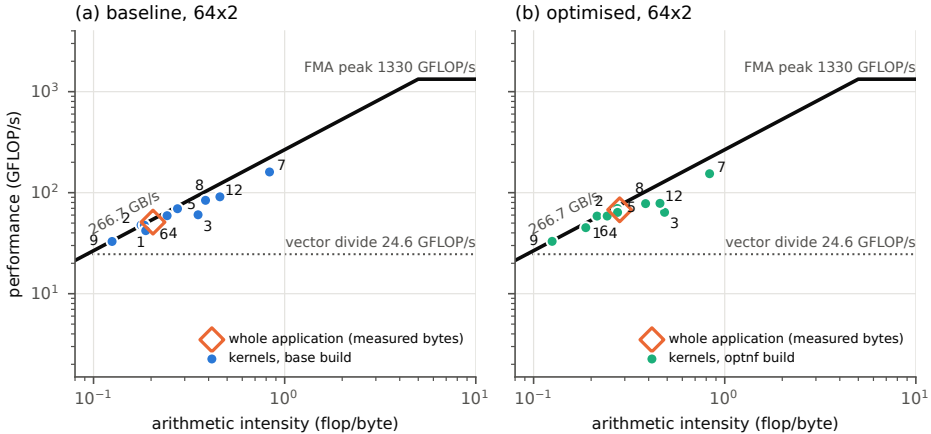
baseline job 34798, optimised job 34939; time = PAPI region time (max over threads); Reset and Revert have no C region any more

Backup: correctness gate, all runs

kernels	build	runs	deviation (%)	layouts (all identical)	gate
c	base	22	0.5726209E-02	128x1, 4x32, 64x2, 1x128, 32x4	ok
fortran	base	12	0.1787088E-01	128x1, 4x32, 64x2	ok
c	opt	6	0.3835202E-01	64x2, 1x128, 4x32, 32x4, 128x1	MISMATCH
fortran	opt	1	0.1787088E-01	64x2	ok
c	flags	1	0.3835202E-01	64x2	MISMATCH
c	cell	1	0.3835202E-01	64x2	MISMATCH
c	mom	1	0.3835202E-01	64x2	MISMATCH
c	optnf	6	0.5726209E-02	64x2, 1x128, 4x32, 32x4, 128x1	ok
fortran	optnf	1	0.1787088E-01	64x2	ok
c	pdv	6	0.5726209E-02	64x2, 1x128, 4x32, 32x4, 128x1	ok
fortran	pdv	1	0.1787088E-01	64x2	ok

Kinetic-energy deviation on `clover_bm.in` as reported by CloverLeaf; gate = `qa_check.py` against the archived baseline value (jobs 34729 and 34796).

Backup: roofline before and after



university of groningen

Backup: baseline layouts, THP and placement

kernels	layout	wall (s) median (min–max)	DRAM (TB)	bandwidth (GB/s)	busiest/ mean	vs. hw roof (%)
c	64x2	11.32 (11.24–11.33)	2.83	250	1.00	94
c	1x128	26.91 (17.44–29.20)	2.99	111	1.66	42
c	4x32	13.00 (12.76–13.21)	2.87	221	1.01	83
c	32x4	11.46 (11.41–11.47)	2.79	244	1.00	91
c	128x1	11.66 (11.63–11.77)	2.95	253	1.00	95
c	1x128, THP off	13.31 (13.28–13.92)	2.83	212	1.00	80
fortran	64x2	9.33 (9.33–9.42)	2.35	252	1.00	95
fortran	1x128	20.63 (18.40–25.92)	2.44	118	1.45	44
fortran	4x32	10.88 (10.78–11.22)	2.38	219	1.00	82
fortran	32x4	9.46 (9.44–9.52)	2.30	243	1.00	91
fortran	128x1	9.76 (9.66–9.77)	2.46	252	1.00	94

THP = 512 MB pages on this system. With one rank, first touch places whole arrays on one NUMA domain; with four or more ranks every rank touches its own data and THP has no effect.

Backup: branch-free upwind loop (T3) in `advec_mom`

```
const double nf  = node_flux[IDXN(j,k)];
const int    neg = (nf < 0.0);                  /* flow in -j: upwind j+2, donor j+1, downwind j */
const double v_m1 = vel1[IDXN(j-1,k)], v_0 = vel1[IDXN(j,k)], v_p1 = vel1[IDXN(j+1,k)], v_p2 = vel1[IDXN(j+2,k)];
const double v_up  = neg ? v_p2 : v_m1;         /* selects replace the branch and the indirect loads */
const double v_don = neg ? v_p1 : v_0;
const double v_dwn = neg ? v_0  : v_p1;
const double m_don = neg ? node_mass_pre[IDXN(j+1,k)] : node_mass_pre[IDXN(j,k)];
const double sigma = fabs(nf)/m_don;
const double vdiffuw = v_don - v_up, vdiffdw = v_dwn - v_don;
const double auw = fabs(vdiffuw), adw = fabs(vdiffdw);
const double wind = (vdiffdw <= 0.0) ? -1.0 : 1.0;
const double lim  = wind*MIN(width*((2.0-sigma)*adw/width+(1.0+sigma)*auw/w_dif)/6.0, MIN(auw,adw));
const double limiter = (vdiffuw*vdiffdw > 0.0) ? lim : 0.0;   /* evaluated, then selected (-fno-trapping-math) */
mom_flux[IDXN(j,k)] = (v_don + (1.0-sigma)*limiter)*nf;
```

Same arithmetic in the same order as the branchy original: bit-identical results; all 13 loops of the file vectorise.